Koha Staff Schedule Plugin Documentation

This document provides a reference for the structure and functional logic of the **kohastaffschedule** plugin.

Directory Structure

```
kohastaffschedule/
├── Koha/
│ └── Plugin/
│ └── Com/
│ └── McKinneyLibrary/
│ ├── kohastaffschedule.pm
│ └── kohastaffschedule/
│ ├── dashboard.tt
│ └── api/
│ └── routes.pl
```

File Breakdown

1. kohastaffschedule.pm (Backend Engine)

This is the core plugin module. It handles plugin registration, database installation (creating `plugin_ks_assignments` and other tables), and permission injection (adding the `manage_schedule` permission to Koha).

2. dashboard.tt (Frontend Template)

This file acts as the UI bridge. It loads Koha's header and footer and initializes the JavaScript environment by mounting data onto `window.KohaScheduleConfig`. This allows the frontend to access staff and branch data directly.

3. api/routes.pl (Database Controller)

This file provides the REST API endpoints that replace your previous Supabase calls. It uses `C4::Context->dbh` to interact directly with the Koha MariaDB database, ensuring all scheduling data remains native to your ILS.

Plugin Logic Flow

- **Request:** An admin opens the Koha staff client and runs the **kohastaffschedule** tool.

- **Perl Initialization:** The module validates the user's `manage_schedule` permission, gathers staff/branch data from Koha, and feeds it into the template.
- **Frontend Render:** The dashboard renders in the browser, initializing a JavaScript environment with the passed data.
- **Interaction:** The admin interacts with the schedule grid. When a shift is created or edited, JavaScript sends a `fetch()` request to the API layer (`routes.pl`).
- **Database Persistence:** The API layer updates the MariaDB tables, keeping the schedule synced natively within the library's primary database.